AGENT EVOLUTION · NOTEBOOK 8 OF 8

Version 8 — Structured Output: a Contract for the Supervisor

A guide to `V8_structured_output_nlq_agent.ipynb`: how the agent stops answering in prose and starts returning a fixed, typed result, with the model's judgment and the database facts kept apart until Python joins them.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · `ToolStrategy` · Pydantic · LangGraph state and `Command`

Tools: lookups · `validate_query` · `run_query` (captures rows) · Middleware: unchanged from Version 7

1. What changes from Version 7
2. The end-to-end workflow
3. Two contracts
4. How `ToolStrategy` works
5. Bad or missing decisions
6. `run_query` captures the rows
7. Middleware, unchanged
8. Reading the trace: `facts_from_state`
9. `assemble()` and the four shapes
10. One call, step by step
11. What changed in the prompt
12. Shape is not truth
13. How this maps to the project
14. Key takeaways

1 What changes from Version 7

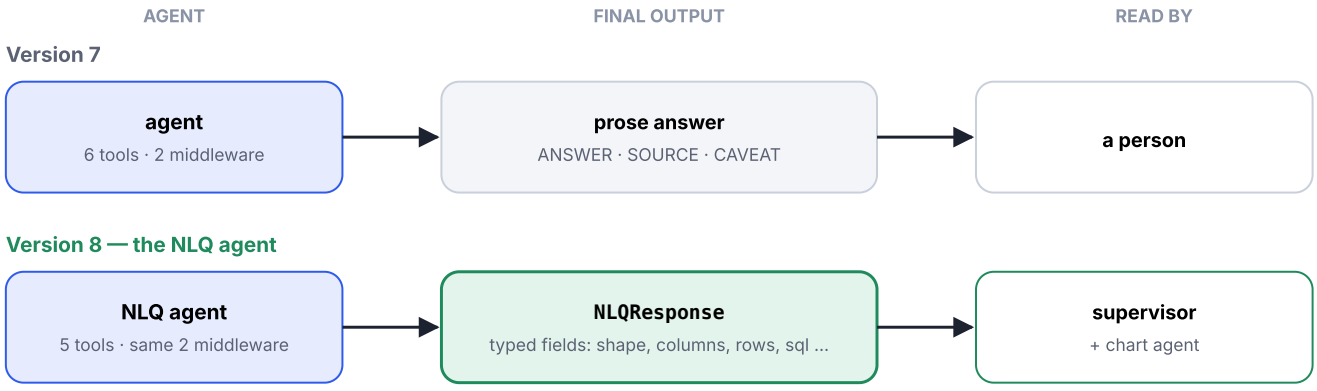
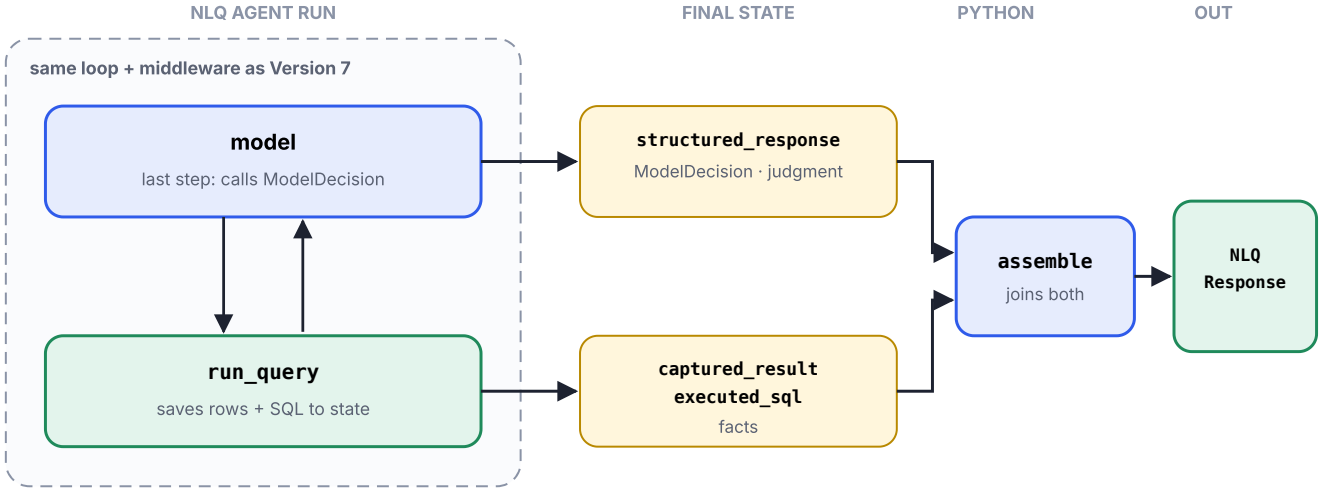


Figure 1. The agent becomes one part of a bigger system. Its reader is now a program.

	Version 7	Version 8
Final output	prose	`NLQResponse` (Pydantic model)
How the model ends	writes text	calls a `ModelDecision` tool (`ToolStrategy`)
Where rows come from	the model retypes them	`run_query` saves them in state
`run_query` returns	text	a `Command` : text for the model + rows for the app

`web_search`	yes	**removed**: the NLQ agent answers internal data only
Prompt	OUTPUT FORMAT (prose)	**FINAL DECISION** (fill `ModelDecision`)
Middleware	budget + cleaner	unchanged

2 The end-to-end workflow



Top lane: what the model judged. Bottom lane: what the database returned. They meet only in Python.

Figure 2. Judgment and facts travel on separate lanes and are joined by plain Python.

1. **The agent runs the same loop as Version 7**: lookups, validation, `run_query`, both middleware.
2. **`run_query` saves its rows and SQL into agent state** (`captured_result`, `executed_sql`), as well as giving the model its usual text.
3. **The model's last step is a call to a `ModelDecision` tool.** LangChain checks it and stores it as `structured_response`.
4. **`assemble()`**, a plain Python function, joins the decision and the facts into an `NLQResponse`.
5. **The supervisor and chart agent** read that object's fields directly.

> **The model never writes the rows.** In Version 7 the model copied numbers from the tool result into its answer, and could copy them wrong. In Version 8 the rows go from `run_query` to state to `NLQResponse` without passing through the model's writing at all.

3 Two contracts

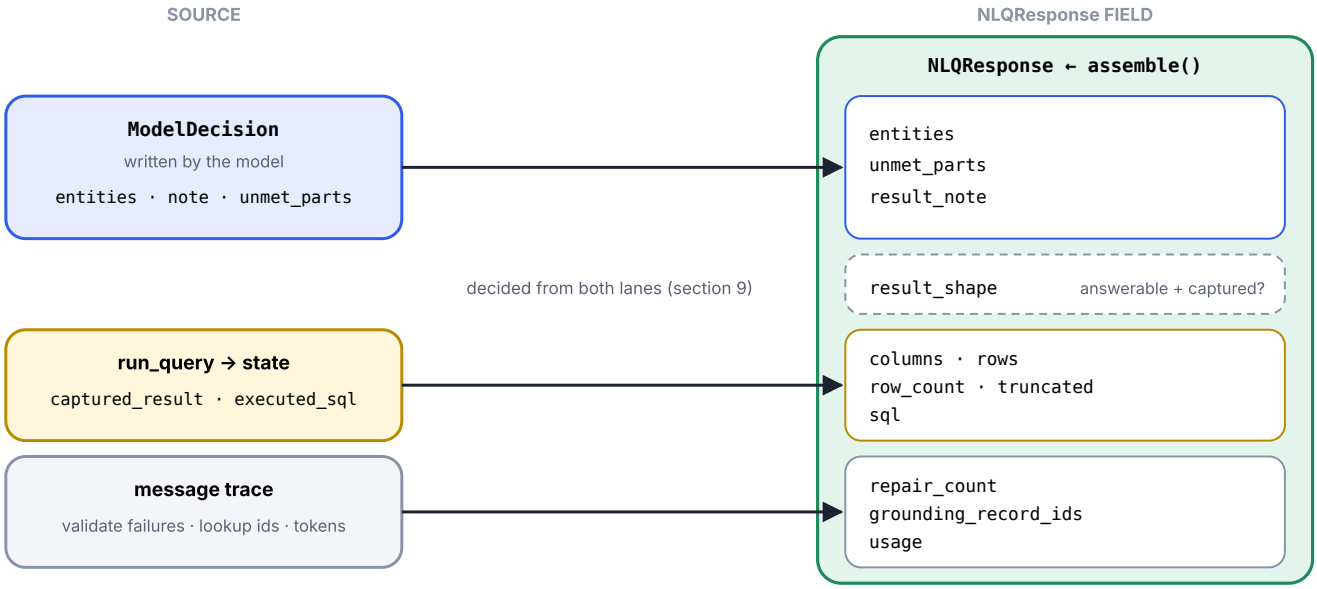


Figure 3. Every `NLQResponse` field has one source. The model's fields are judgments only.

Model	Written by	Holds
`ModelDecision`	the model, through `ToolStrategy`	`entities`, `answerable`, `note`, `unmet_parts` — judgments
`NLQResponse`	Python, in `assemble()`	`result_shape`, `columns`, `rows`, `row_count`, `truncated`, `sql`, the decision fields, `grounding_record_ids`, `repair_count`, `usage`

`ModelDecision` has **no** `rows` and **no** `sql` field. That is the whole idea. The model has nowhere to write a table or a query, so it cannot invent one. Its job shrinks to what only a model can do: judge whether the question was answered, name the entities, and add a caveat.

> `result_shape` is decided by both inputs: `answerable` from the model, and whether a result was captured. Section 9 shows the four outcomes.

4 How `ToolStrategy` works

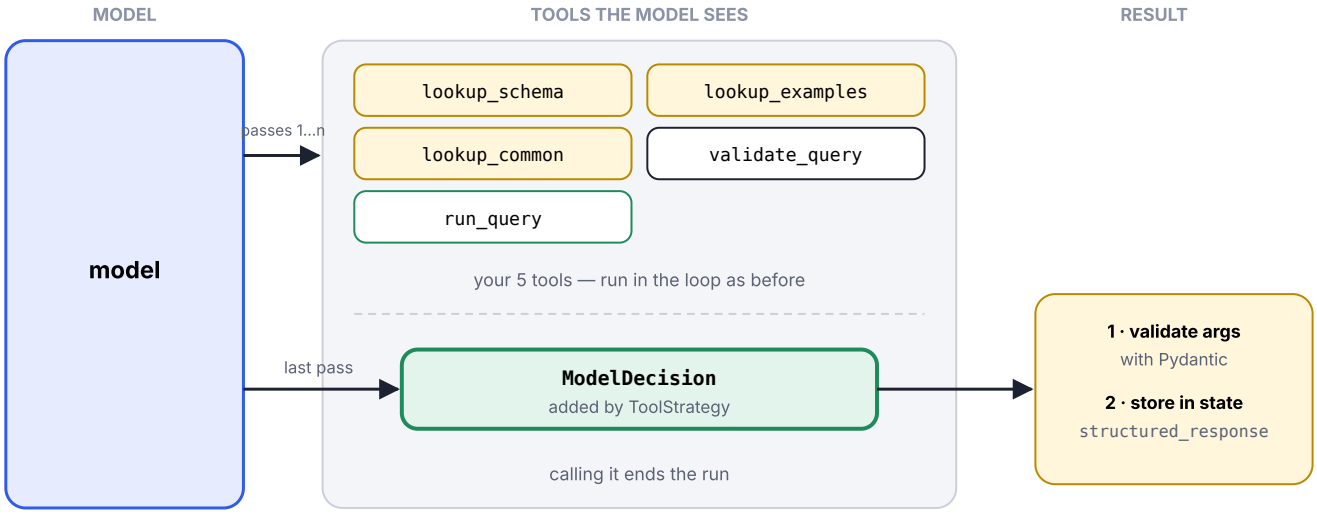


Figure 4. `ToolStrategy` gives the model one extra tool. Its arguments are the structured answer.

```
agent = create_agent(
    model=model,
    tools=[lookup_schema, lookup_examples, lookup_common, validate_query, run_query],
    system_prompt=SYSTEM_PROMPT,
    state_schema=NLQState,
    response_format=ToolStrategy(schema=ModelDecision),
    middleware=[SQLBudgetMiddleware(max_sql_calls=3),
ToolOutputMiddleware(max_output_chars=5000)],
    name="nlq",
)
```

Models are already good at one structured task: filling in a tool's arguments. `ToolStrategy` uses that. It turns the `ModelDecision` class into a tool schema and adds it to the tools the model sees. When the model is done, it "calls" `ModelDecision` with its judgment as the arguments.

Checked on the real library:

- **The graph is unchanged:** still `__start__ → model ⇄ tools → __end__`.
- **The result gains `structured_response`**, holding a real `ModelDecision` object, next to `messages`, `sql_call_count`, `captured_result` and `executed_sql`.
- **The last message is now a `ToolMessage`:** `Returning structured response: entities=[…] answerable=True …`. It is no longer the model's answer, so the `result["messages"][-1].content` habit from Versions 1–7 no longer gives you the answer. Read `structured_response` instead.

5 Bad or missing decisions

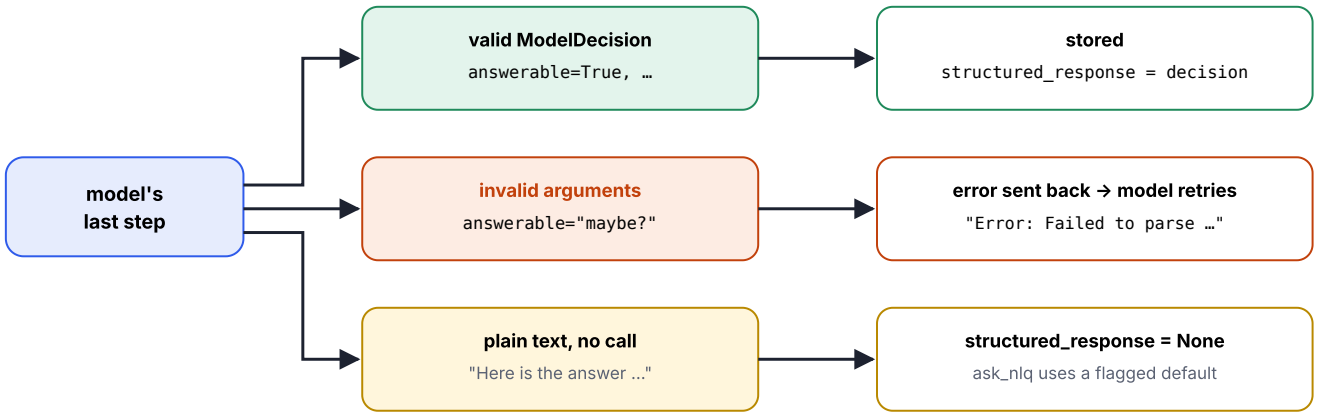


Figure 5. The three ways a run can end. All three were tested on the real library.

- **Invalid arguments are retried.** In a test, the model first sent `answerable="maybe?"` and `entities="x"`. LangChain replied with a `ToolMessage` starting `Error: Failed to parse structured output for tool 'ModelDecision': … 2 validation errors …`. The model tried again with valid values, and that decision was stored.
- **No call at all leaves `structured_response` as `None`.** If the model ends with plain text, the run finishes normally but has no decision.

A bug this guide caught. The first draft of `ask_nlq()` read `state["structured_response"]` and passed it straight to `assemble()`. With `None` that crashed. The notebook now uses `state.get(...)` and, when it is missing, a default `ModelDecision` whose note says no decision was returned. The result is a `not_executed` or `table` response, flagged in `result_note`, never an exception.

6 `run_query` captures the rows

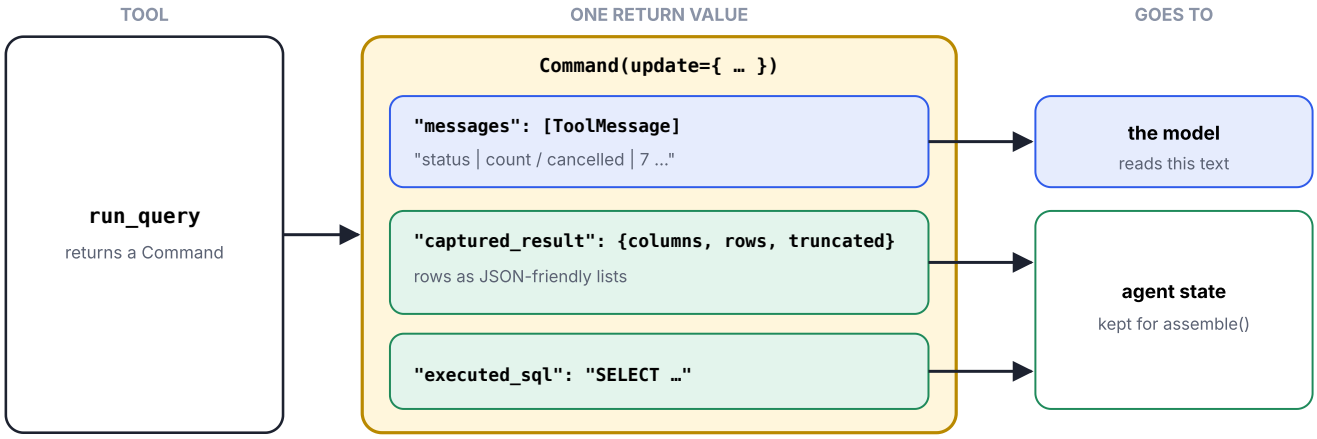


Figure 6. One return value, two audiences: text for the model, structured rows for the application.

```
class NLQState(SQLBudgetState):
    captured_result: NotRequired[dict | None]
    executed_sql: NotRequired[str]

@tool
def run_query(sql: str, runtime: ToolRuntime) -> Command:
    ...
    return reply(format_rows(columns, rows, truncated),
                 {"captured_result": captured, "executed_sql": sql_or_reason})
```

- **`NLQState` extends the budget's state** with two keys. It is passed to `create_agent(state_schema=NLQState)`.
- **`runtime: ToolRuntime` is injected by LangChain and hidden from the model.** The schema the model sees is still only `{"sql": {"type": "string"}}` (checked). The tool uses `runtime.tool_call_id` to build its `ToolMessage`.
- **Values are made JSON-friendly:** `Decimal` → `float`, dates → ISO text, so the supervisor can send the result anywhere.
- **Only successful queries are captured.** On `REFUSED` or `SQL error` the tool returns just the message, so the last good result in state is kept.

7 Middleware, unchanged

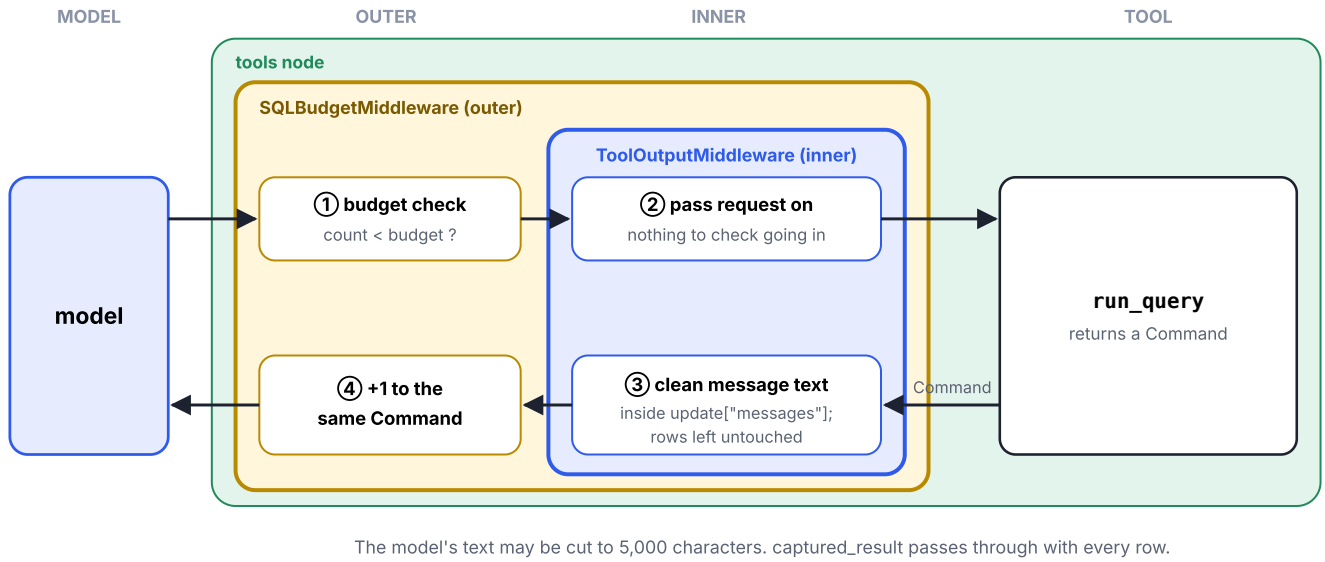


Figure 7. The Version 7 middleware works unchanged on a `Command` result.

Both middleware classes were written in Versions 6 and 7 to handle a `Command`, and that pays off here:

- **`ToolOutputMiddleware`** finds the `ToolMessage` inside `update["messages"]` and cleans only its text. `captured_result` and `executed_sql` pass through unchanged.
- **`SQLBudgetMiddleware`** adds `sql_call_count: +1` to the same `Command`.

> **Two views of one result.** On a wide 50-row query, the model may see only the first 5,000 characters plus the "do not infer missing rows" marker. The application still gets all 50 rows in `captured_result`. The cut protects the model's context without losing data for the supervisor.

8 Reading the trace: `facts_from_state`

NLQResponse field	Read from	How
`rows`, `columns`, `truncated`	`state["captured_result"]`	as saved by `run_query`
`sql`	`state["executed_sql"]`	the last SQL that ran successfully
`repair_count`	`validate_query` ToolMessages	count those starting `VALIDATION FAILED`
`grounding_record_ids`	`lookup_*` ToolMessages	every `| id=… |` in the results, in order, no duplicates
`usage`	every `AIMessage`	sum of `usage_metadata` tokens; count of model calls

None of these fields asks the model anything. They are read from what actually happened in the run. The fixed prefixes chosen in earlier versions make this possible: `VALIDATION FAILED` (Version 4) and `RESULT n | id=… |` (Version 3) are easy to find in the trace.

A test run of the notebook's agent produced `repair_count=1` (one failed validation), two grounding ids from `lookup_schema`, and `llm_calls=5`.

9 `assemble()` and the four shapes

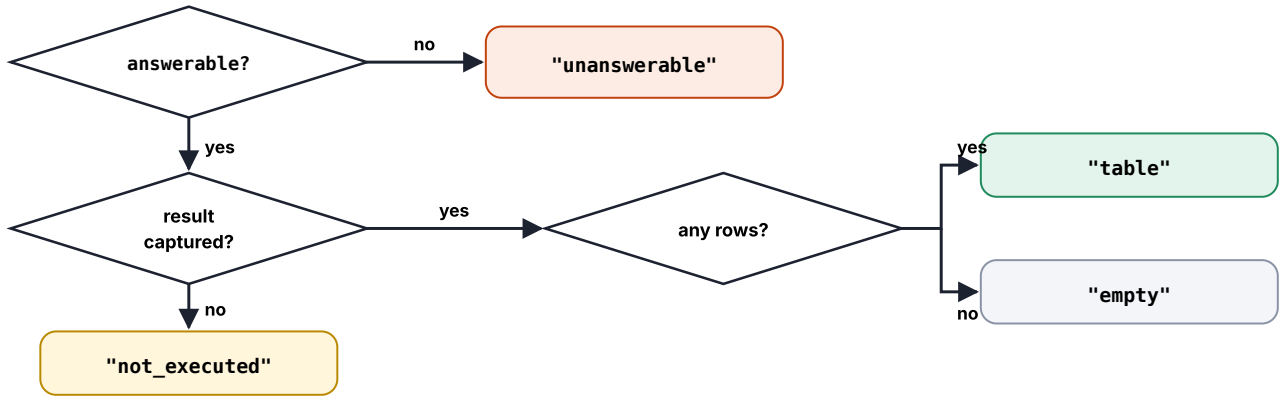


Figure 8. `assemble()` picks one of four shapes. Each tells the supervisor something different.

Shape	Means	Supervisor can...
`table`	a query ran and returned rows	show or chart the rows
`empty`	a query ran correctly and matched nothing	say “no results”, which is a real answer
`unanswerable`	the model judged the data cannot answer it	show `result_note` and stop
`not_executed`	no query result was captured	retry, or route the question elsewhere

`assemble()` is a plain function of plain inputs, so Part A of the notebook runs it offline with mock data, no keys needed. If the captured result was truncated, it also adds “Result capped at N rows; more rows exist.” to `result_note`.

> In Version 7 all four cases arrived as sentences, and a program had to guess which one it was reading. Now each is a different value of one field.

10 One call, step by step

```
state["messages"] (shortened):
  HumanMessage("Which products generated the most revenue last month?")
  AIMessage(lookup_schema(...))      → ToolMessage("RESULT 1 | id=... ")
  AIMessage(validate_query(...))     → ToolMessage("VALIDATION FAILED: ...")
  AIMessage(validate_query(...))     → ToolMessage("VALIDATION PASSED: ...")
  AIMessage(run_query(...))          → ToolMessage("product | revenue ...")
  AIMessage(ModelDecision(entities=[...], answerable=True, note="..."))
  ToolMessage("Returning structured response: entities=[...] ...")

state["structured_response"]  → ModelDecision(...)
state["captured_result"]      → {"columns": [...], "rows": [[...], ...], "truncated": False}
state["executed_sql"]         → "SELECT ..."
state["sql_call_count"]       → 1
```

And the `NLQResponse` built from that state, in the shape it took in a test run of the notebook's agent:

```
{
  "result_shape": "table",
  "columns": ["status", "count"],
  "rows": [["cancelled", 7]],
  "row_count": 1,
  "truncated": false,
  "sql": "SELECT status, COUNT(*) FROM orders GROUP BY 1",
  "entities": ["cancelled"],
  "unmet_parts": [],
  "grounding_record_ids": ["nlq-schema-rec0", "nlq-schema-rec1"],
  "result_note": "...",
  "repair_count": 1,
  "usage": {"input_tokens": 500, "output_tokens": 50,
            "total_tokens": 550, "llm_calls": 5, ...}
}
```

Notice what the model contributed: `entities` and the note. Every other field came from the tools and the trace.

11 What changed in the prompt

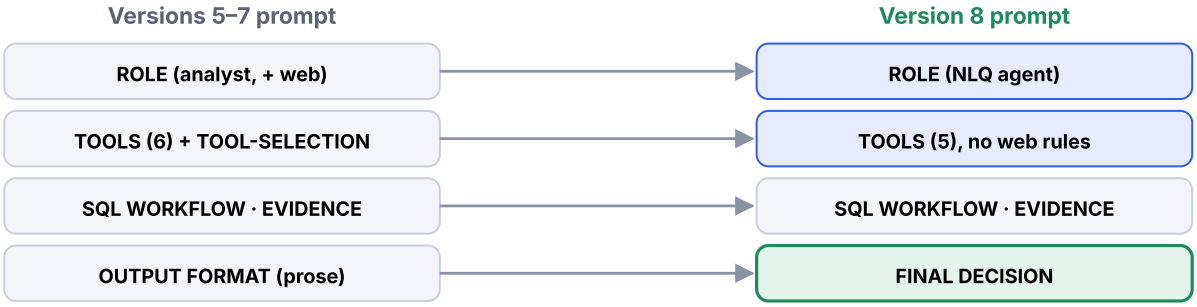


Figure 9. The SQL workflow is kept word for word. The role, the tool list and the ending change.

```
FINAL DECISION
When you have finished using tools, return a ModelDecision:
- answerable: false only if the schema or data cannot answer the question.
- entities: the main business entities the answer is about, as they appear in the query
result.
- note: one or two sentences on the definitions used and any material caveat.
- unmet_parts: parts of the question you could not answer; empty if none.
Do not put SQL, column lists, or result rows in the ModelDecision. The application
reads those directly from run_query.
```

- **ROLE:** now “the NLQ agent … Your result is passed to a supervisor and a chart agent.”
- **One new SQL rule:** “The last successful run_query result is the table the application returns, so make sure your final query is the one that answers the question.” Without it, the model might run a final check query and leave the wrong table in state.
- **FINAL DECISION** replaces the prose OUTPUT FORMAT and explains each field in one line.

> The prompt was changed carefully and only where the output changed. The SQL workflow lines are word for word the same as in Versions 5–7, so query behaviour carries over.

12 Shape is not truth

Pydantic can check	Pydantic cannot check
`answerable` is a boolean	whether the question really was answerable
`entities` is a list of strings	whether those are the right entities
`result_shape` is one of four values	whether the SQL answered the right question
`rows` is a list of lists	whether the business definition was correct

Structured output guarantees the **shape** of the answer, not its **correctness**. That is why the earlier layers stay: the schema and definitions from lookups (Version 3), SQL validation (Version 4), the SQL budget and clean output (Versions 6–7). And it is why rows and SQL are taken from the tool path, not from the model's text.

> **Why split the contract at all?** (1) Clear jobs: the model judges, tools capture, Python assembles. (2) Reliable data: rows never pass through the model's writing. (3) A fixed interface: the supervisor reads the same fields every time. (4) Distinct failures: `empty`, `unanswerable` and `not_executed` mean different things and are handled differently.

13 How this maps to the project

This notebook	The project's NLQ agent
`run_query`	`execute_sql`
`ask_nlq()` reads the final state	`core.py` reads `update["structured_response"]` from stream events
`assemble()`	`assemble.py`
`SQLBudgetMiddleware`, `ToolOutputMiddleware`	`BudgetMiddleware`, `GuardMiddleware`, `GroundingMiddleware`
`ModelDecision`, `NLQResponse`	the same schemas

The eight notebooks have built, one capability at a time, the shape of a production NLQ agent:

```
V1 prompt anatomy → V2 run_query → V3 knowledge lookups → V4 SQL validation
→ V5 routing → V6 budget middleware → V7 output middleware → V8 structured contract
```

14 Key takeaways

1. **The reader is now a program,** so the output must have fixed, typed fields.
2. **Two contracts:** `ModelDecision` holds the model's judgment; `NLQResponse` holds the final payload.
3. **The model has no field for rows or SQL,** so it cannot invent them.
4. **`ToolStrategy` adds a `ModelDecision` tool.** Calling it ends the run; LangChain validates the arguments and stores them in `structured_response`.
5. **Invalid arguments are sent back and retried; a missing call leaves `None`.** Handle the `None` case.
6. **The last message is now a `ToolMessage`.** Read `structured_response`, not `messages[-1]`.
7. **`run_query` returns a `Command`** with text for the model and rows for state. `ToolRuntime` is injected and hidden from the model.
8. **The Version 7 middleware works unchanged.** The model's text can be cut; `captured_result` keeps every row.
9. **Metadata comes from the trace**, not the model: repairs, grounding ids, token usage.
10. **Four shapes, four meanings:** `table`, `empty`, `unanswerable`, `not_executed`.
11. **Shape is not truth.** Validation, lookups and checks against the trace stay essential.

> **Next topic:** Model Wrapping.